Restaurant Suite Scenarios And Images

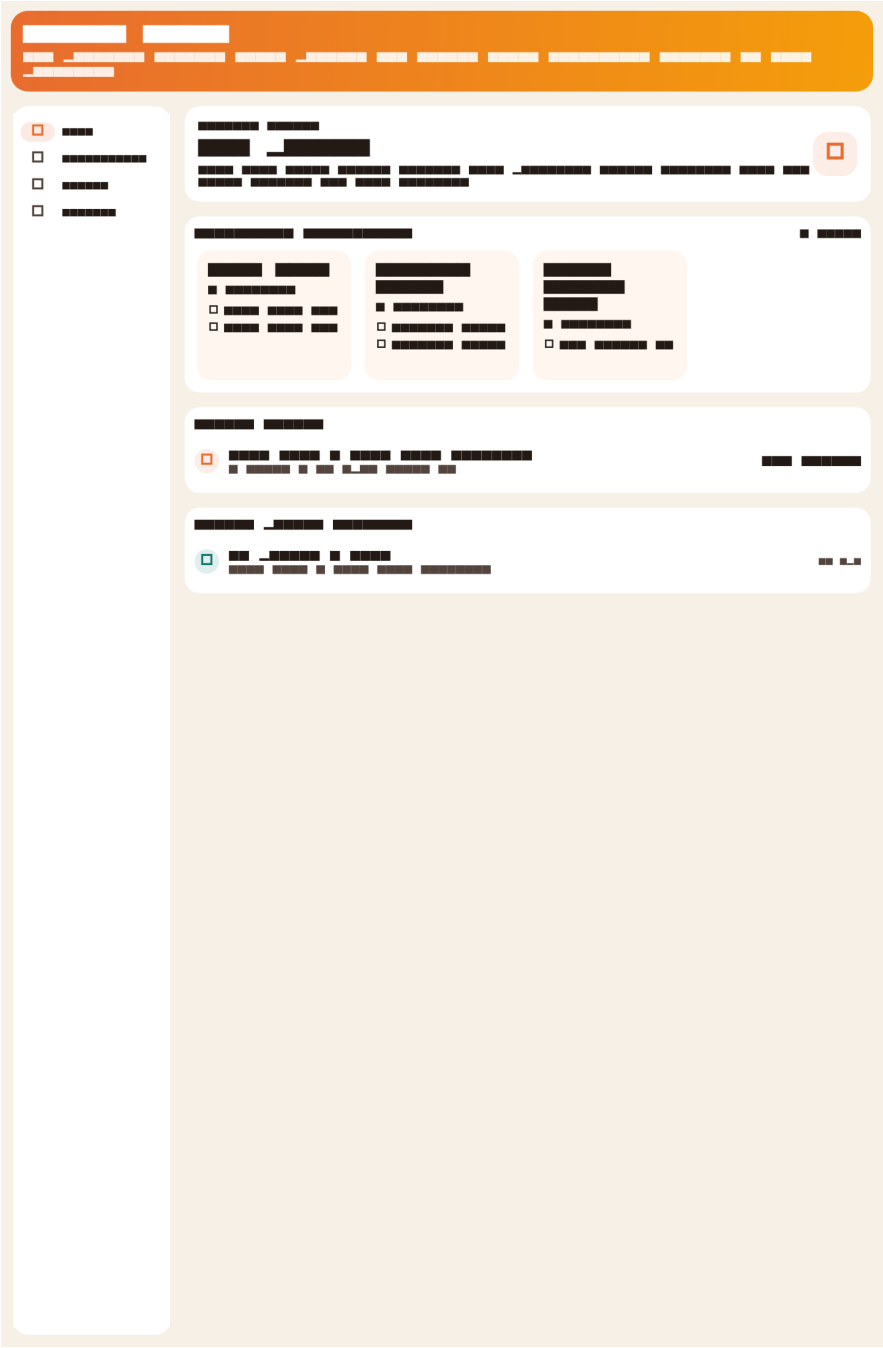
This guide combines the live dashboard captures, generated Flutter workspace screenshots, and the operational flow cards already produced from the API walkthrough.

Flutter images in this document come from widget goldens generated on April 1, 2026 from the current code in `factory_app`. Dashboard images come from the live owner dashboard capture set already in the repository.

Scenario 1: Returning Customer Dines In And Earns Loyalty

Step 1. Customer opens the app and checks order memory, restaurants, and points

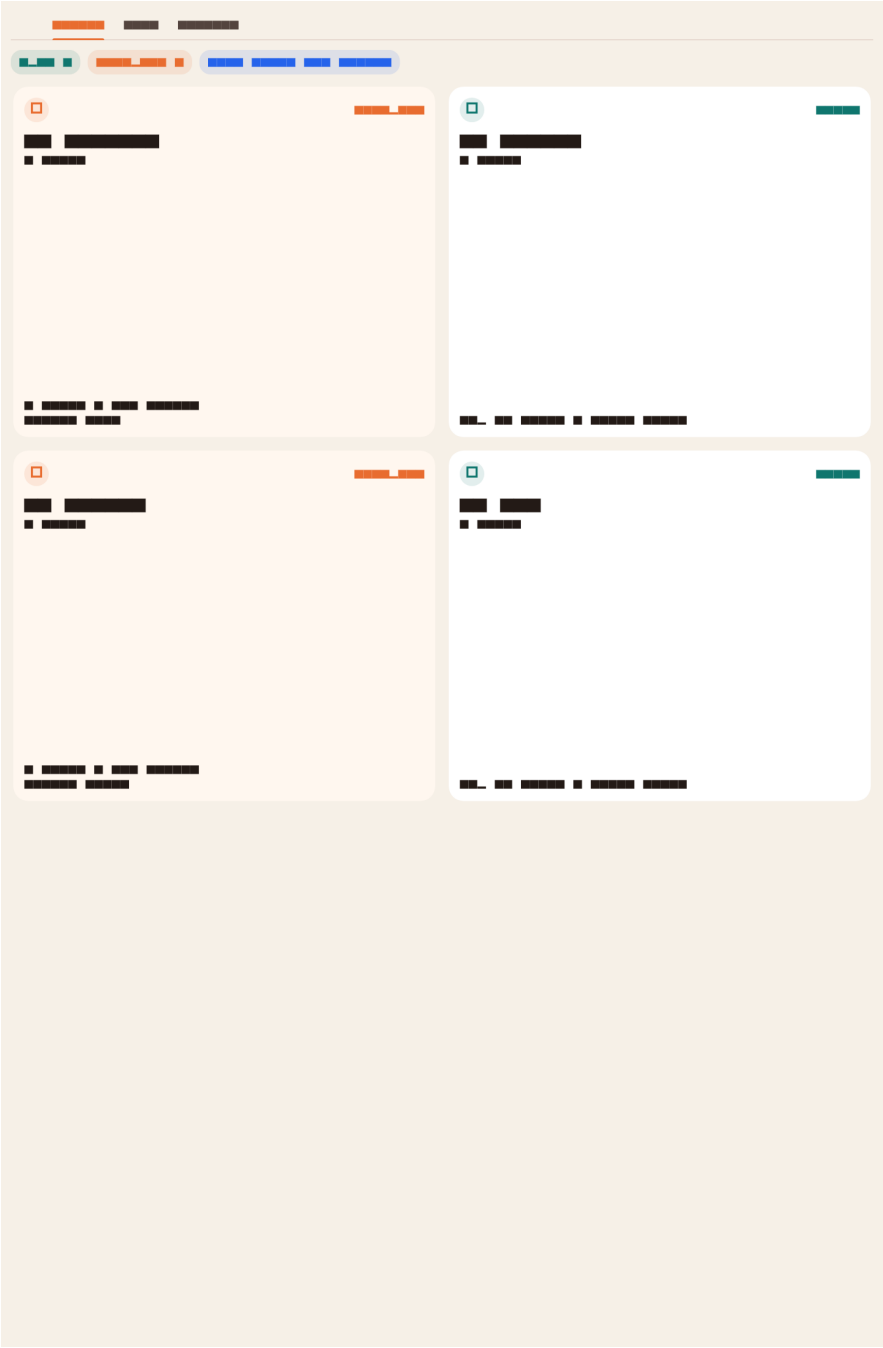
The customer app is designed like a warm marketplace homepage: registered restaurants, previous orders, and loyalty points are visible immediately.



Customer app home with restaurants, order history, and rewards visibility.

Step 2. Waiter sees the floor in a table-first layout

The waiter app prioritizes table occupancy, table value, and guest context, which is the fastest mental model for dine-in operations.



Waiter workspace with live open and occupied tables.

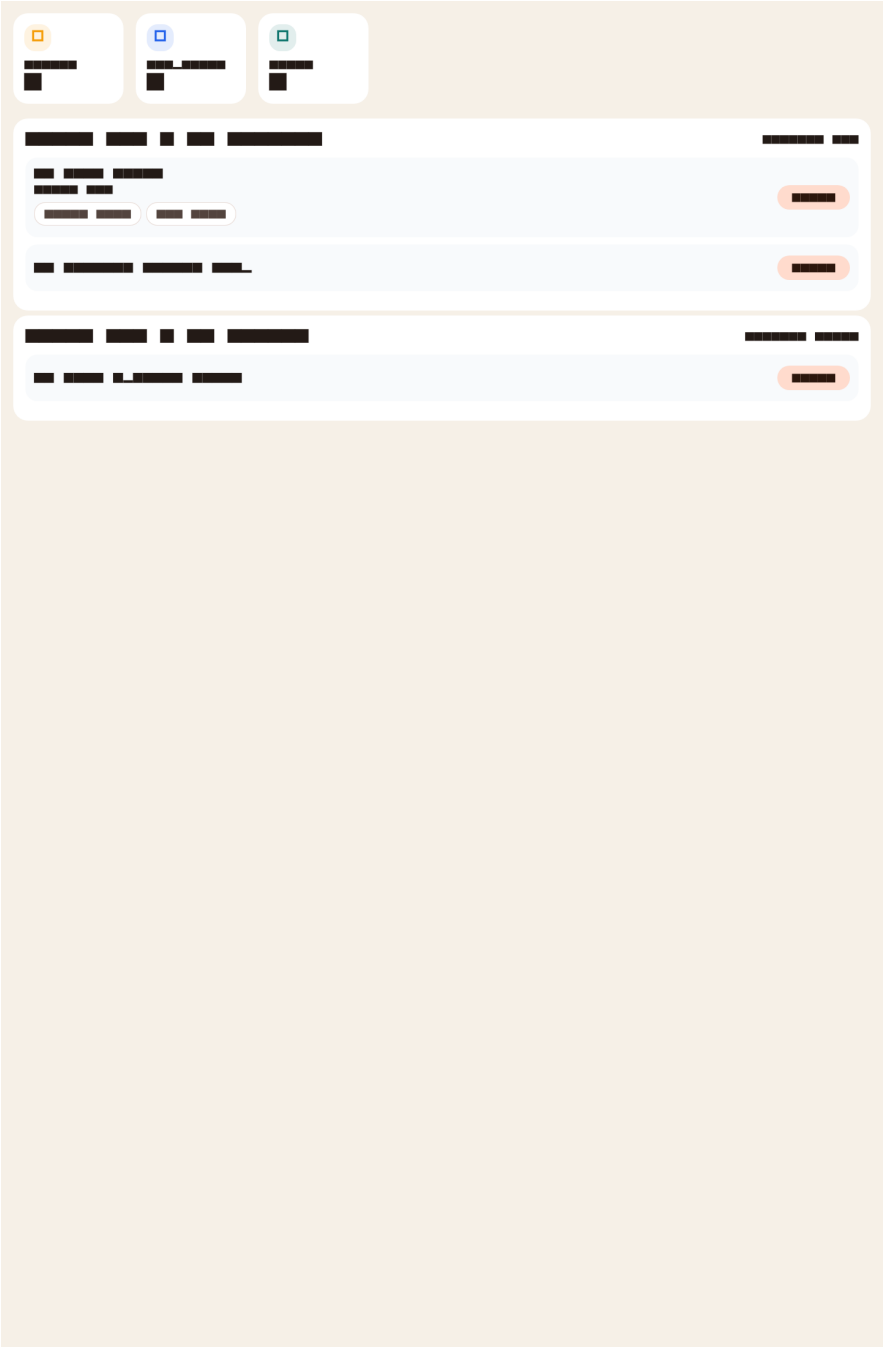
Step 3. Waiter opens the active table, attaches customer phone, adds notes, answers, and modifiers

This closes a meaningful gap in the old repo state because dine-in orders can now be tied to a real customer profile for history and loyalty visibility.

Waiter order detail with customer phone, current items, and menu composer.

Step 4. Kitchen receives the ticket and advances item states

Queued, preparing, and ready states are visible per item. Kitchen can advance each line item without waiting on a full-order status change.



Kitchen board with live queued, preparing, and ready tickets.

Step 5. Cashier settles the order with one or multiple tenders

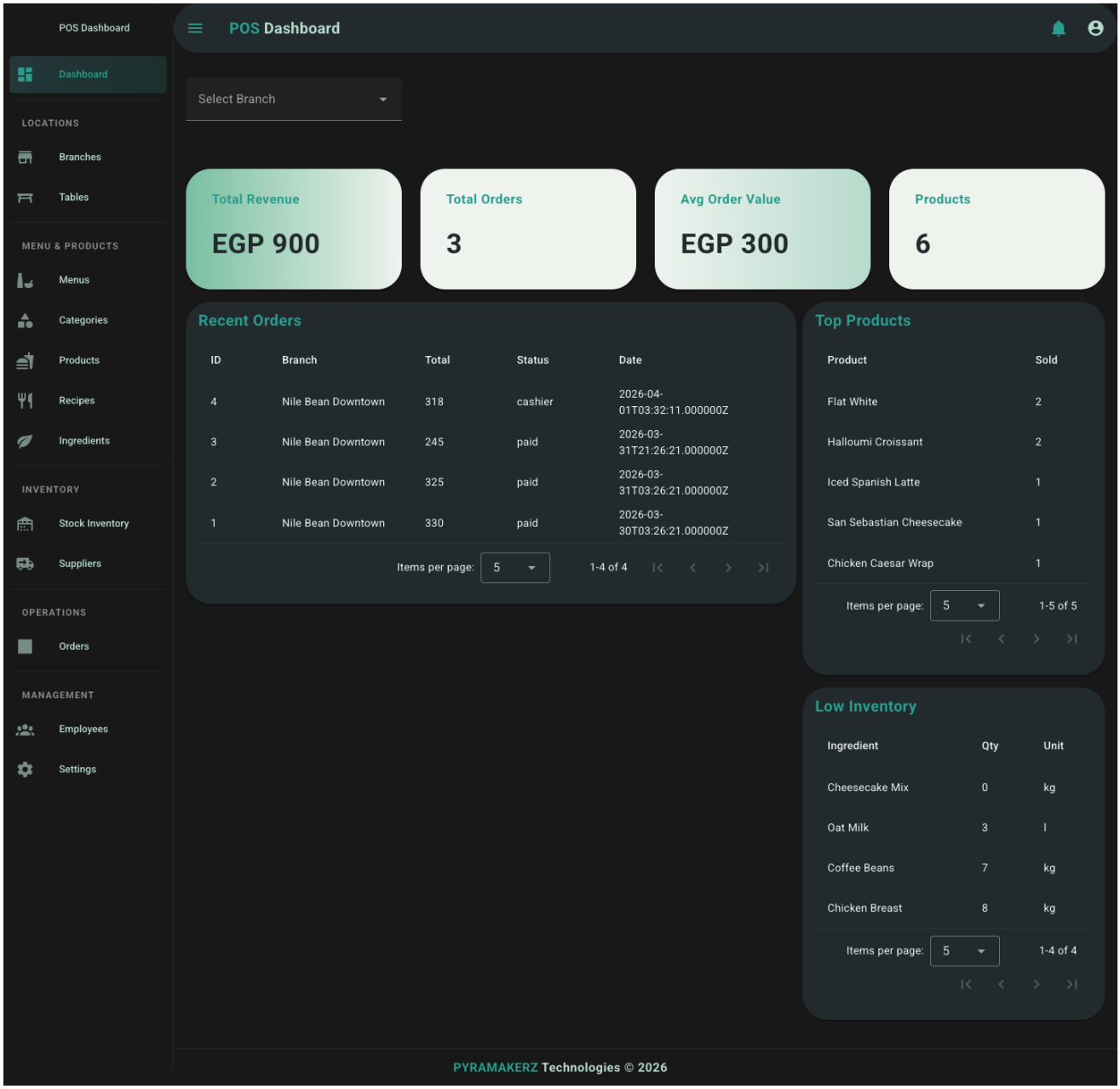
The cashier workspace supports multiple tenders in one settlement flow, which directly addresses a visible gap against Foodics-style dine-in bundles.

Cashier queue and split-payment settlement workspace.

Step 6. Owner and shareholders see the result in branch-level analytics

The owner workspace surfaces revenue, cashier queue, KDS backlog, branch ranking, and loyalty member count. The live dashboard capture below shows the web side of the same operational flow.

Flutter owner control center.



Live web dashboard after the walkthrough payment step.

Scenario 2: Real Dine-In Exception Handling

Quantity Change

The waiter can reduce an item quantity after submission. The API keeps history and recalculates totals.

STEP 04 Waiter App

Change Quantity After The Order Was Sent

Generated operations visual based on the implemented API flow in this repository. The waiter, kitchen, and cashier frontend code is not present here, so this image documents the observed system behavior rather than a literal UI screenshot.

ACTION

- Guest decides to keep only one coffee.
- Waiter calls `PATCH /api/mobile/order-items/7/refund-change` with `action=change` and `quantity=1`.

OBSERVED STATE

- Item `#7` quantity changes from `2` to `1`.
- Item status becomes `changed`.
- Order subtotal and total are recalculated.

OWNER DASHBOARD IMPACT

- Orders screen total drops for order `#4`.
- The owner dashboard Recent Orders panel updates to the new lower amount.

Observed intermediate total after quantity change: `453 EGP` before dessert refund.

POS RESTAURANT DOCUMENTATION ASSET

Refund One Item

The waiter can refund an item and preserve auditability instead of silently editing the order line.

STEP 05 Waiter App

Refund An Item That Should Not Be Served

Generated operations visual based on the implemented API flow in this repository. The waiter, kitchen, and cashier frontend code is not present here, so this image documents the observed system behavior rather than a literal UI screenshot.

ACTION

- Guests defer dessert.
- Waiter calls `PATCH /api/mobile/order-items/9/refund-change` with `action=refund`.

OBSERVED STATE

- Item `#9` becomes `refunded` and its `kds_status` becomes `refunded`.
- Refunded amount is recorded on the item.
- Refunded dessert drops out of active KDS output.

OWNER DASHBOARD IMPACT

- Orders screen total drops again.
- Dashboard KPIs still remain unchanged because the order is not yet `paid`.

Observed order total after change + refund: `318 EGP`.

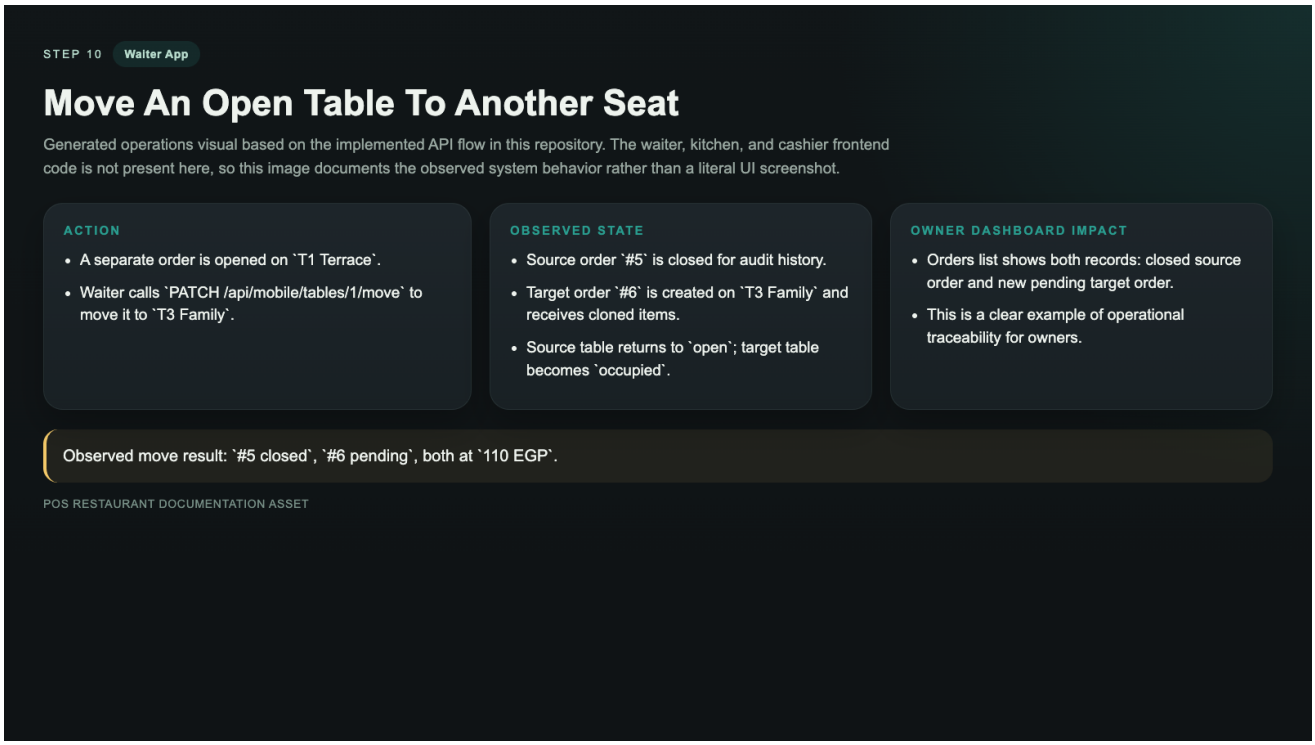
POS RESTAURANT DOCUMENTATION ASSET

Reopen After Settlement

Paid orders can be reopened, which is important for late add-ons and floor corrections.

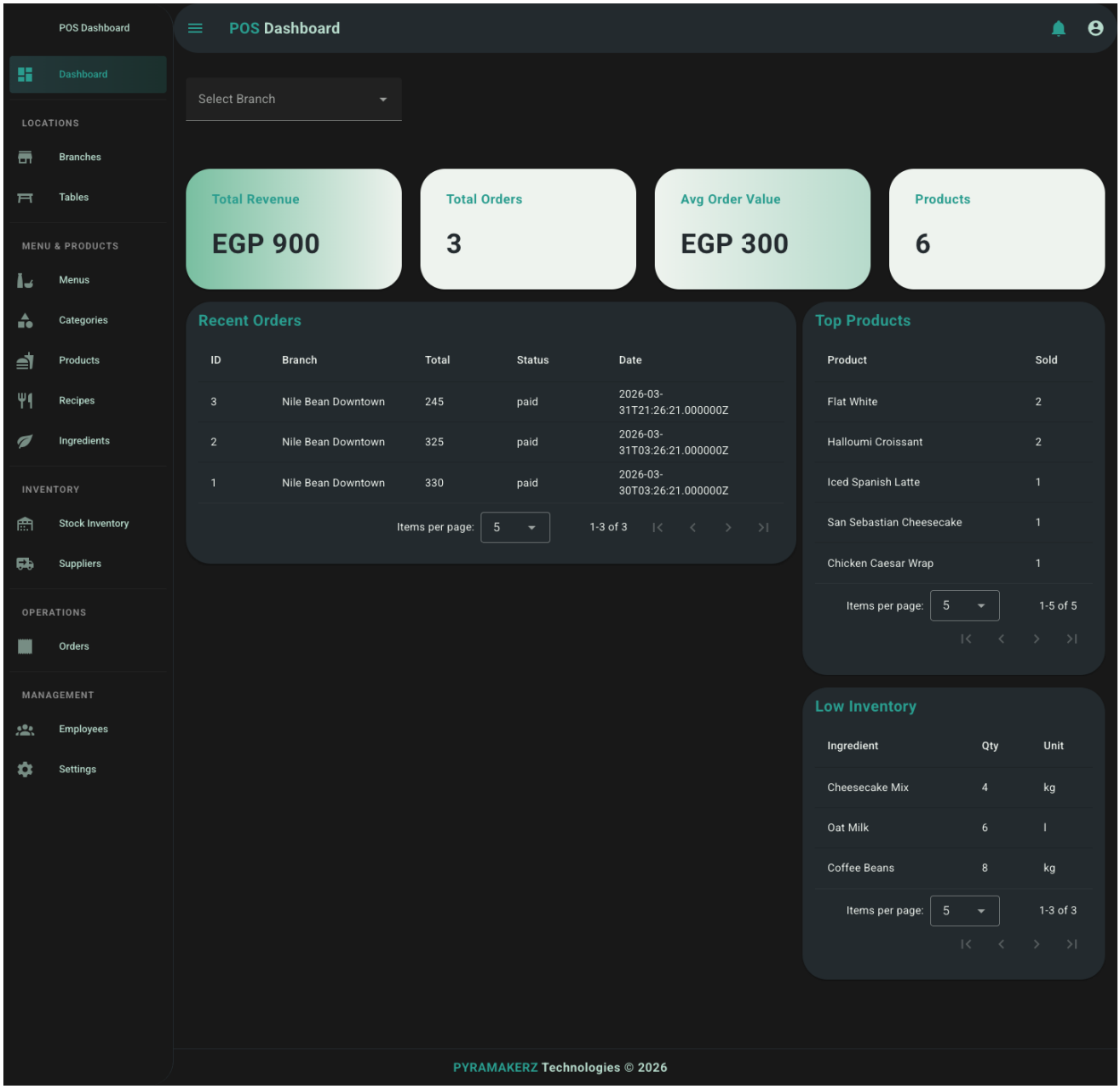
Move Table

The system supports audit-safe table moves by closing the source order and cloning active items into the target table order.



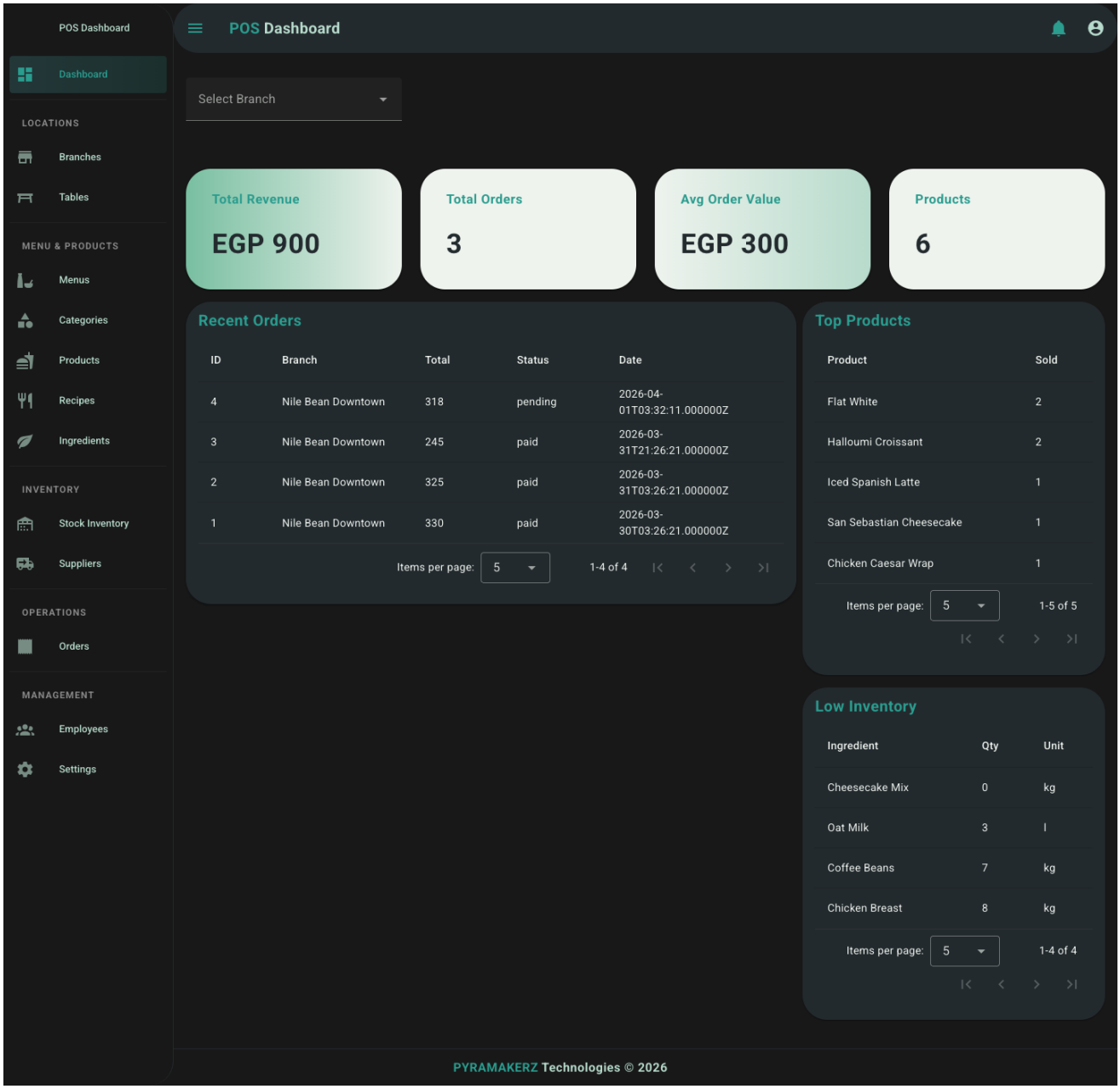
Scenario 3: Dashboard Effects During The Same Service Window

Before Service



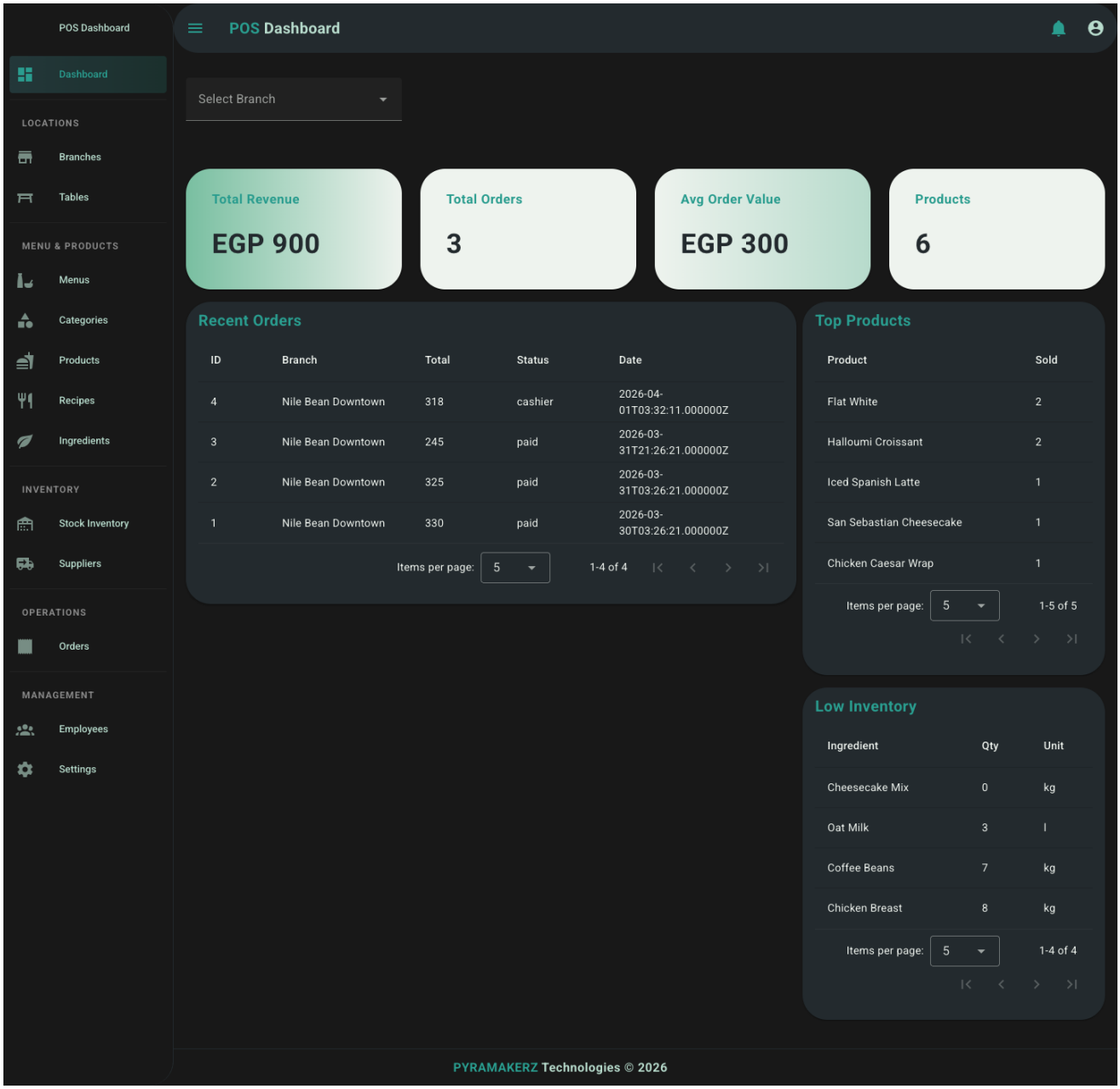
Baseline owner KPIs before the walkthrough flow started.

After Order And Adjustments



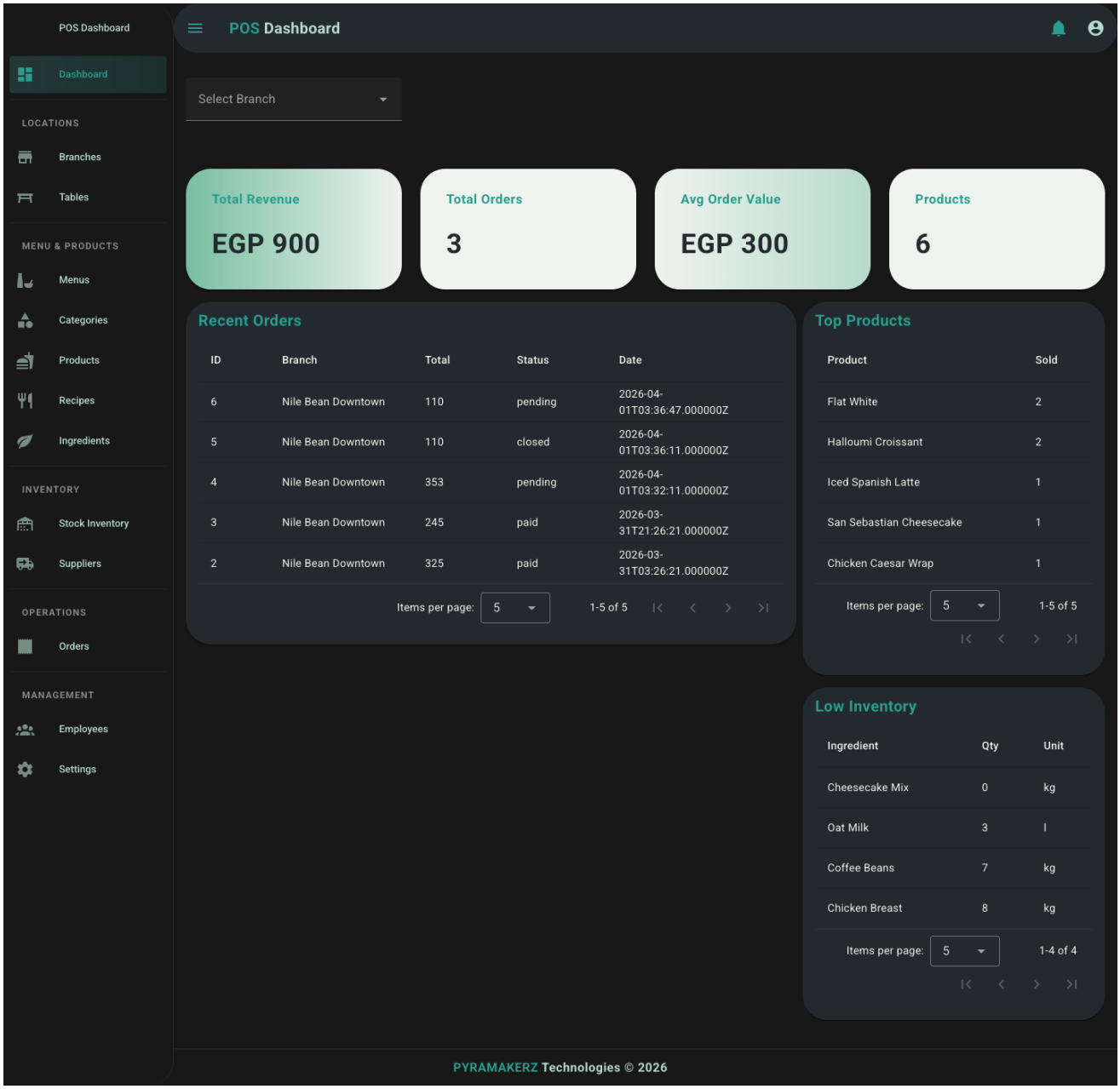
Revenue pipeline and recent orders reflect order creation and item adjustments.

After Payment



Payment data and recent activity update after settlement.

After Table Move



Order history preserves the source and target table timeline.

Recommended Next Scenario Packs

- Large family table splits the bill across cash plus card plus wallet.
- Busy brunch service with one kitchen bottleneck and one cashier bottleneck.
- VIP repeat guest arrives, waiter recognizes phone, and owner dashboard later measures loyalty lift.
- Multi-branch operator compares lunch shift performance between branches and drills into payment mix.